§12. 输入输出流 – C++方式的文件操作

要求：

1、安装UltraEdit软件（附件已给，版本虽旧，但够用），学会使用16进制方式查看文件，并掌握ASCII及16进制查看间的切换

★ 已安装VSCode的也可通过相关插件进行16进制方式的查看(VSCode某种情况下会自动做格式转换或字符集转换，要注意!!!)

★ 也可以使用其它编辑软件，但不建议NotePad++

2、完成本文档中所有的测试程序并填写运行结果，从而体会二进制与十进制文件的差异，掌握与文件有关的流函数的正确用法

3、题目明确指定编译器外，缺省使用VS2026即可

★ 如果要换成其他编译器，可能需要自行修改头文件适配（不强制要求Linux，但建议试一试）

★ 部分代码编译时有warning，不影响概念理解，可以忽略

4、直接在本文件上作答，写出答案/截图（不允许手写、手写拍照截图）即可；填写答案时，为适应所填内容或贴图，允许调整页面的字体大小、颜色、文本框的位置等

★ 贴图要有效部分即可，不需要全部内容

★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可

★ 不允许手写在纸上，再拍照贴图

★ 允许在各种软件工具上完成（不含手写），再截图贴图

★ 如果某题要求VS+gcc的，则如果两个编译器运行结果一致，贴VS的一张图即可，如果不一致，则两个图都要贴

5、转换为pdf后提交

6、10月29日前网上提交本次作业（在“文档作业”中提交）

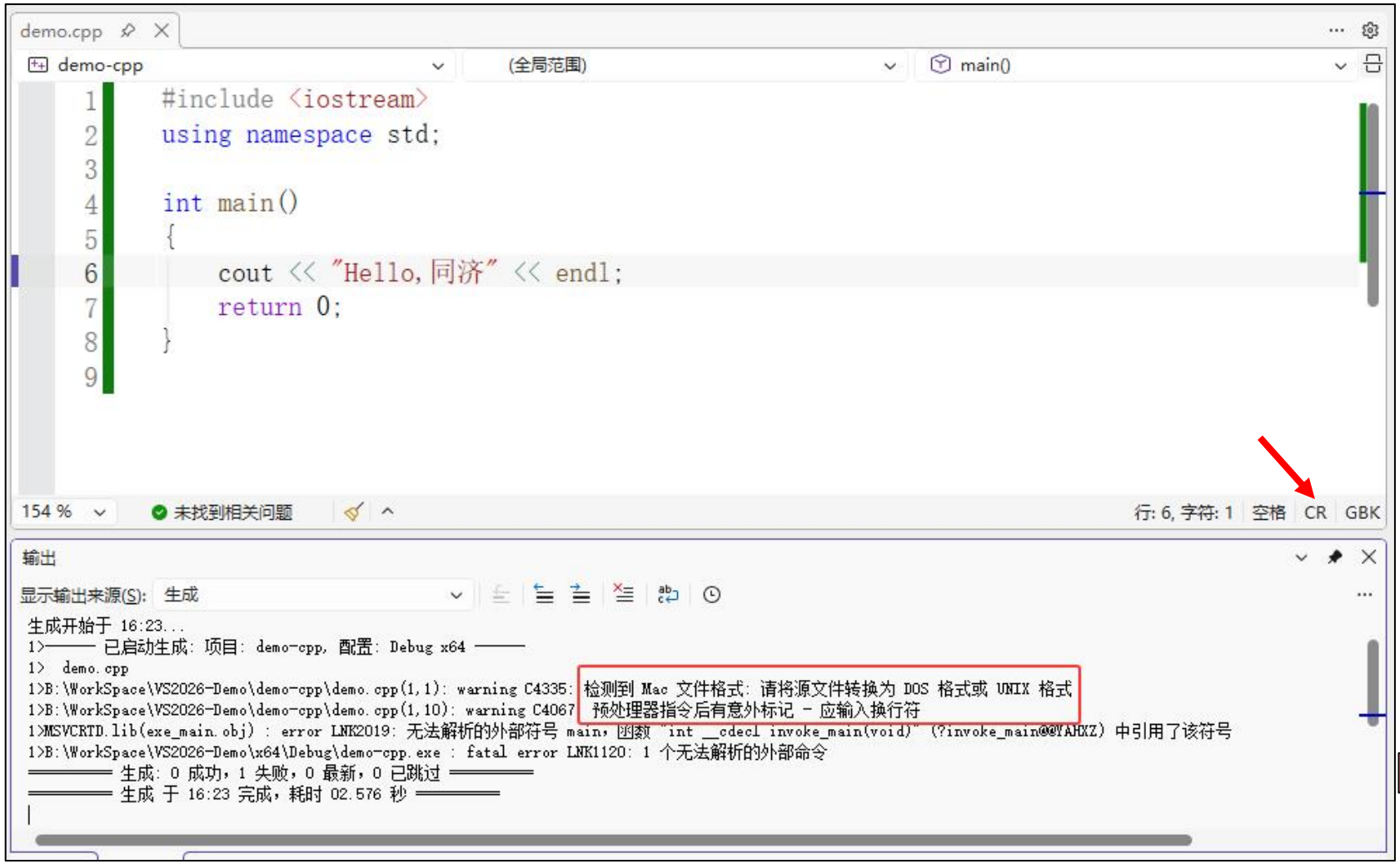
特别说明：

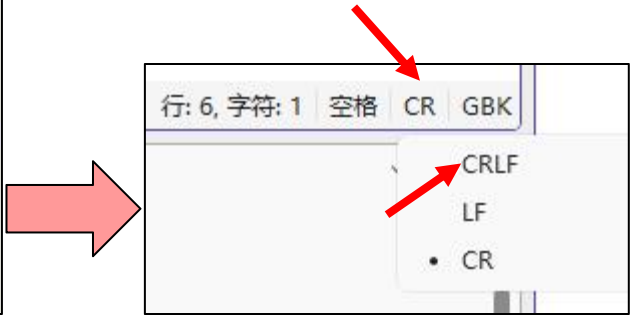
★ 因为篇幅问题，打开文件后均省略了是否打开成功的判断，这在实际应用中是不允许的

§12. 输入输出流 – C++方式的文件操作

注意：

附1：用WPS等其他第三方软件打开PPT，将代码复制到VS2026中后，如果出现类似下面的编译报错，则观察源程序编辑窗的右下角是否为CR，如果是，单击CR，在弹出中选择CRLF，再次CTRL+F5运行即可

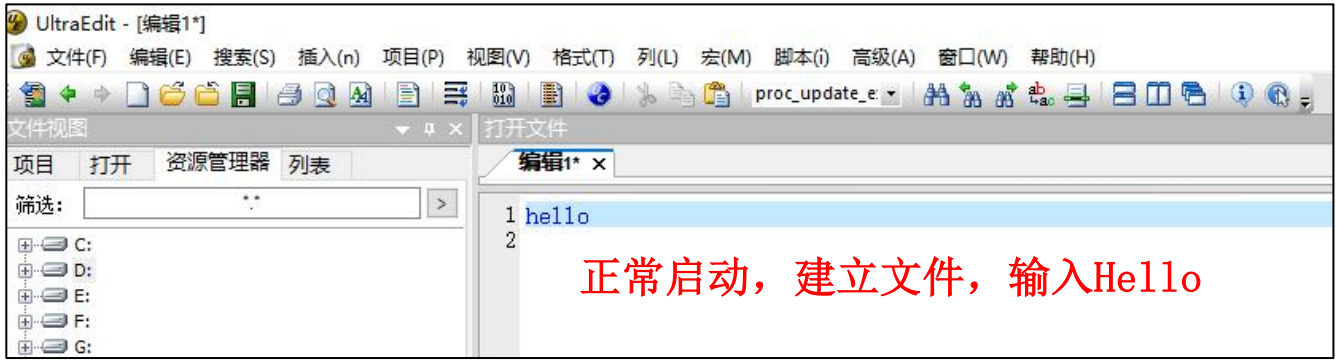




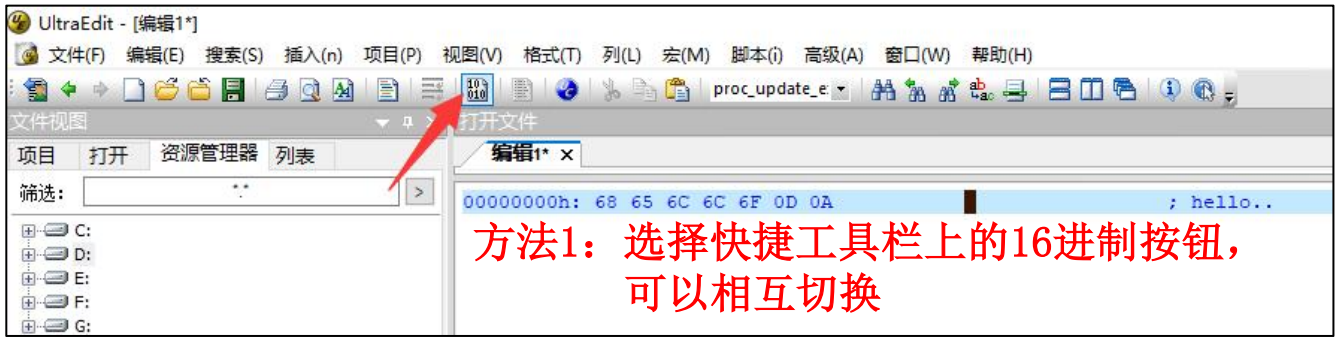
§12. 输入输出流 – C++方式的文件操作

注意：

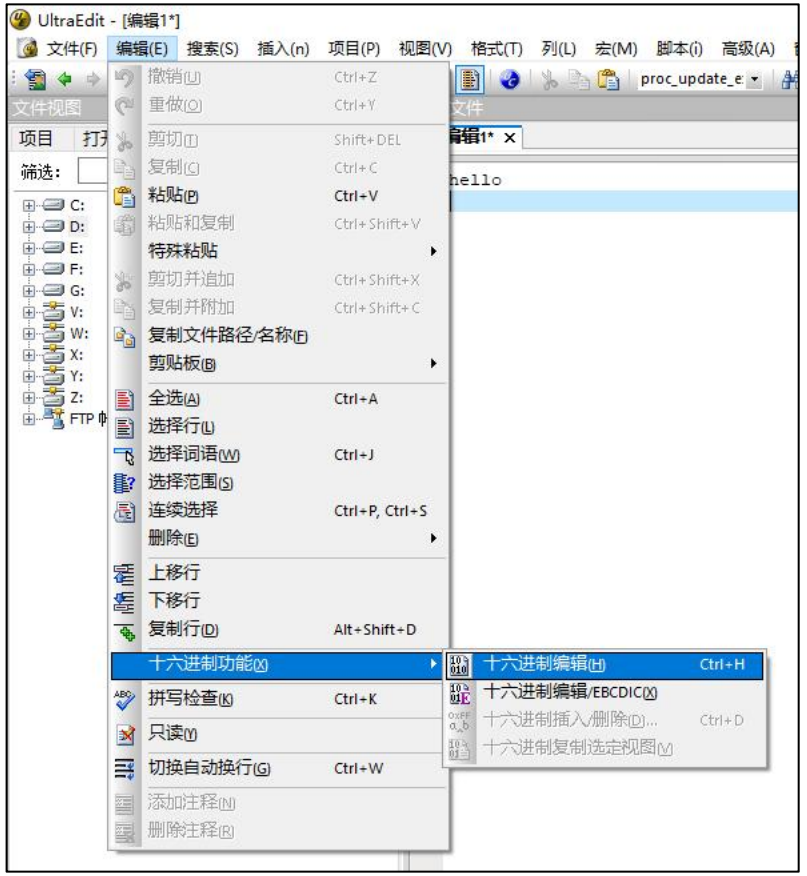
附2：附件给出的UltraEdit查看文件的16进制形式的方法（三种）



正常启动，建立文件，输入Hello



方法1：选择快捷工具栏上的16进制按钮，
可以相互切换



方法2：”编辑”－”十六进制功能”菜单，
可以相互切换

方法3：Ctrl + H 快捷键可以相互切换

§12. 输入输出流 - C++方式的文

已作答

例1：十进制方式写

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);

    out << "hello" << endl;  //去掉endl后再次运行

    out.close();

    return 0;
}
```

Windows下运行，out.txt是7字节（有endl的情况），用UltraEdit的16进制方式打开的贴图

Windows下运行，out.txt是5字节（无endl的情况），用UltraEdit的16进制方式打开的贴图

注：VSCode等其他软件的16进制查看方式的截图也可以，下同

说明：十进制（文本）方式下 **endl** 的 **'\n'** 被 **CRT** 翻译成 **0D 0A**，故多 **1** 个字节。





§12. 输入输出流 - C++方式的文

已作答

例2：二进制方式写

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);

    out << "hello" << endl;  //去掉endl后再次运行

    out.close();

    return 0;
}
```

Windows下运行，out.txt是6字节（有endl的情况），用UltraEdit的16进制方式打开的贴图
Windows下运行，out.txt是5字节（无endl的情况），用UltraEdit的16进制方式打开的贴图
综合例1/2，endl在十进制和二进制方式下有无区别？
综合例1/例2：有区别。十进制方式下 endl 写 2 字节（0D 0A），二进制方式下只写 1 字节（0A）。





§12. 输入输出流 - C++方式的文

已作答

例3：十进制方式写，十进制方式读，0D0A(即"\r\n")在Windows下的表现

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    ifstream in("out.txt", ios::in);
    while(!in.eof())
        cout << in.get() << ' ';
    cout << endl;
    in.close();
    return 0;
}
```

Windows下运行，输出结果是：

说明：0D 0A在Windows的十进制方式下被当做1个字符处理，值是10（'\n'）。

答：输出为 **104 101 108 108 111 10 -1**（末尾的 **-1** 是 **while(!in.eof())** 多读一次 **EOF** 所致）。



§12. 输入输出流 - C++方式的文

已作答

例4：十进制方式写，二进制方式读，0D0A(即"\r\n")在Windows下的表现

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    while(!in.eof())
        cout << in.get() << ' ';
    cout << endl;
    in.close();
    return 0;
}
```

Windows下运行，输出结果是：

说明：0D 0A在Windows的二进制方式下被当做2个字符处理，值是13（'\r'）和 10（'\n'）。

答：输出为 **104 101 108 108 111 13 10 -1**。



§12. 输入输出流 – C++方式的文

已作答

例5：十进制方式写，十进制方式读，不同读方式在Windows下的表现

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in);
    in >> str;
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：

说明：in>>str读到空白字符（'\n'）就结束了，'\n'还被留在缓冲区中，因此in.peek()读到了10。

答：输出 5 和 10。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：

说明：in.getline读到'\n'就结束了，'\n'被读掉，因此in.peek()读到了-1（EOF）。

答：输出 5 和 -1。



§12. 输入输出流 - C++方式的文

已作答

例6：二进制方式写，十进制方式读，不同读方式在Windows下的表现

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in);
    in >> str;
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：
说明：in>>str读到空白字符（'\n'）就结束了，'\n'还被留在缓冲区中，因此in.peek()读到了10。
答：输出 5 和 10。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：
说明：in.getline读到'\n'就结束了，'\n'被读掉，因此in.peek()读到了-1（EOF）。
答：输出 5 和 -1。



§12. 输入输出流 - C++方式的文

已作答

例7：二进制方式写，二进制方式读，不同读方式在Windows下的表现

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in | ios::binary);
    in >> str;
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：
说明：in>>str读到空白字符（'\n'）就结束了，'\n'还被留在缓冲区中，因此in.peek()读到了10。
答：输出 5 和 10。

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out | ios::binary);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in | ios::binary);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：
说明：in.getline读到'\n'就结束了，'\n'被读掉，因此in.peek()读到了-1（EOF）。
答：输出 5 和 -1。





§12. 输入输出流 - C++方式的文

已作答

例8：十进制方式写，二进制方式读，不同读方式在Windows下的表现

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in | ios::binary);
    in >> str;
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，输出结果是：

说明：in>>str读到空白字符（'\r'）就结束了，'\r'还被留在缓冲区中，因此in.peek()读到了13。

答：输出 5 和 13。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "hello" << endl;
    out.close();

    char str[80];
    ifstream in("out.txt", ios::in | ios::binary);
    in.getline(str, 80);
    cout << strlen(str) << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

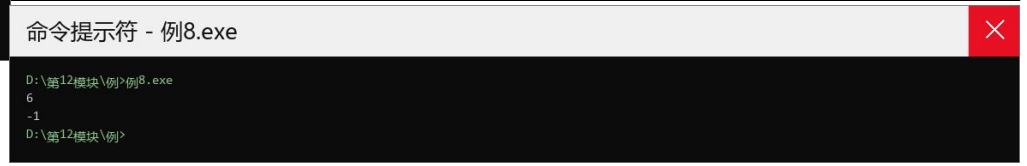
Windows下运行，输出结果是：

说明：

1、in.getline读到'\n'就结束了，'\n'被读掉，因此in.peek()读到了-1（EOF）。

2、strlen(str)是6，最后一个字符是'\r'（0D）

答：输出 6 和 -1（二进制方式不合并 0D0A）。



§12. 输入输出流 – C++方式的文

已作答

例9：用十进制方式写入含\0的文件，观察文件长度

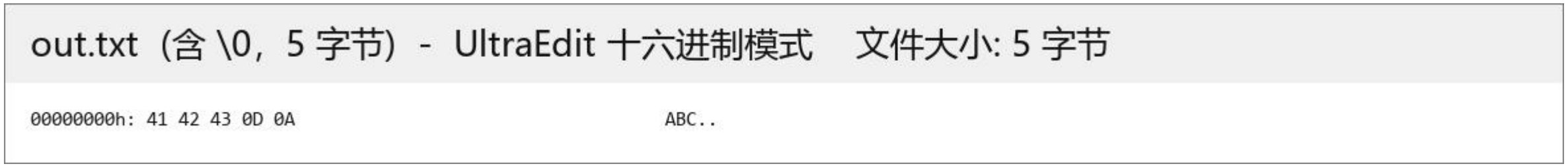
```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\0\x61\x62\x63" << endl;
    out.close();

    return 0;
}
```

Windows下运行，out.txt的大小是5字节，为什么？

说明：**operator<<** 输出 **const char*** 时按 **C** 字符串处理，遇 **'\0'** 即停止，只写入了 **"ABC"**；再加 **endl** 的 **0D 0A**，共 **5** 字节，后面的 **\x61\x62\x63** 没有写入。



§12. 输入输出流 – C++方式的文

已作答

例10：用十进制方式写入含非图形字符(ASCII码32是空格，33-126为图形字符)，但不含\0

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def" << endl;
    out.close();

    return 0;
}
```

Windows下运行，out.txt的大小是20字节，UltraEdit的16进制显示截图为：

说明：字符串本身 **18** 个字符（含 **01 02 1A 09 0B 08 FF 7D** 等非图形字符），再加 **endl** 写出的 **0D 0A**，共 **20** 字节。

out.txt（含非图形字符，20 字节） - UltraEdit 十六进制模式　文件大小: 20 字节

```
00000000h: 41 42 43 01 02 1A 09 0B 08 FF 7D 28 29 2D 3D 64  ABC.......}()-=d
00000010h: 65 66 0D 0A                                      ef..
```

§12. 输入输出流 - C++方式的文

已作答

例11：用十进制方式写入含\x1A(十进制26=CTRL+Z)的文件，并用十进制/二进制方式读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in);
    int c=0;
    while(!in.eof()) {
        in.get();
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：20
输出的c是：6

为什么？

说明：**MSVC** 文本方式把 **0x1A（Ctrl+Z）**当作文件结束符，读到第 **6** 个字节即置 **eofbit**，循环结束。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\xff\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    int c=0;
    while(!in.eof()) {
        in.get();
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：20
输出的c是：21

c的大小比文件大小大1，原因是：二进制方式不认0x1A，20 字节读完后 eof 未置位，循环多执行一次



§12. 输入输出流 – C++方式的文

已作答

例12：用十进制方式写入含\x1A(十进制26=CTRL+Z)的文件，并用十进制不同方式读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in);//不加ios::binary
    int c=0;
    while(in.get()!=EOF) {
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：19
输出的c是：5

为什么？

说明：读到 **0x1A** 时文本方式置 **eof**，**get()** 返回 **EOF**，循环条件不成立退出，只计了前 **5** 个字节。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\x1A\t\v\b\175()-=def"<<endl;
    out.close();

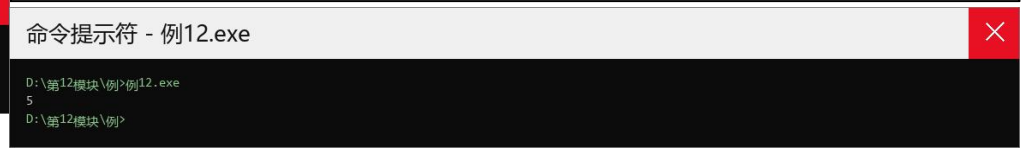
    ifstream in("out.txt", ios::in); //不加ios::binary
    int c=0;
    char ch;
    while((ch=in.get())!=EOF) {
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：19
输出的c是：5

为什么？

说明：仍是 **0x1A** 让文本方式提前结束；本例没有 **0xFF**，**char** 截断问题未暴露，两种写法结果相同。



§12. 输入输出流 – C++方式的文

已作答

例13：用十进制方式写入含\xFF(十进制255/-1，EOF的定义是-1)的文件，并进行正确/错误读取

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\xff\t\v\b\175()-=def"<<endl;
    out.close();

    ifstream in("out.txt", ios::in);//可加ios::binary
    int c=0;
    while(in.get()!=EOF) {
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：19
输出的c是：18

为什么？

说明：文本方式把 0D0A 合并成 0A（少 1 个），0xFF 正常读出，共 18 个字符，第 19 次 get() 返回 EOF 退出。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABC\x1\x2\xff\t\v\b\175()-=def"<<endl;
    out.close();

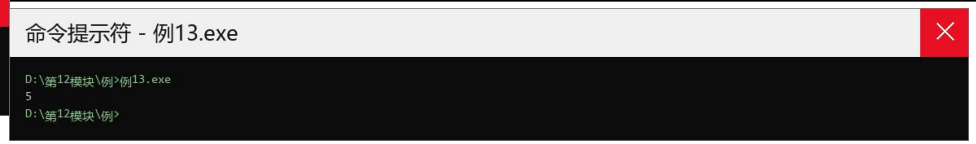
    ifstream in("out.txt", ios::in); //可加ios::binary
    int c=0;
    char ch;
    while((ch=in.get())!=EOF) {
        c++;
        }
    cout << c << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：19
输出的c是：5

为什么？

说明：get() 返回 255，赋给 char 变成 -1，提升为 int 仍是 -1，与 EOF 相等，循环读到 0xFF 即提前结束。



综合例11~例13，结论：当文件中含字符0x1A（Ctrl+Z）时，不能用十进制方式读取，而当文件中含字符0xFF时，是可以用二/十进制方式正确读取的

§12. 输入输出流 - C++方式的文

已作答

例14：比较格式化读和read()读的区别，并观察gcount()/tellg()在不同读入方式时值的差别

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ" << endl;
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[30];
    in >> name;
    cout << '*' << name << '*' << endl;
    cout << int(name[26]) << endl;
    cout << in.gcount() << endl;
    cout << in.tellg() << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：28

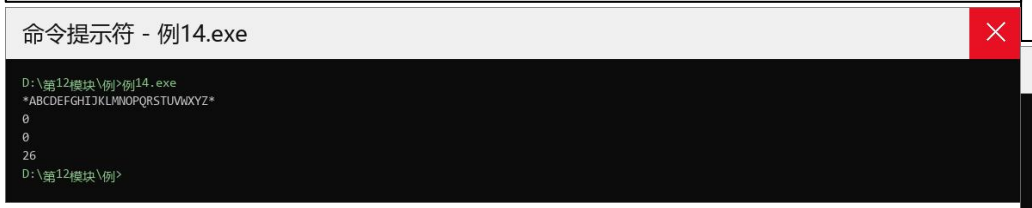
输出的name是：ABCDEFGHIJKLMNOPQRSTUVWXYZ

name[26]的值是：0

gcount()的值是：0

tellg()的值是：26

说明：in >> 方式读入字符串时，和cin方式相同，都是读到空白字符（'\n'）停止，并在数组最后加入一个'\0'。




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ" << endl;
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[30];
    in.read(name, 26);
    cout << '*' << name << '*' << endl;
    cout << int(name[26]) << endl;
    cout << in.gcount() << endl;
    cout << in.tellg() <<endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：28

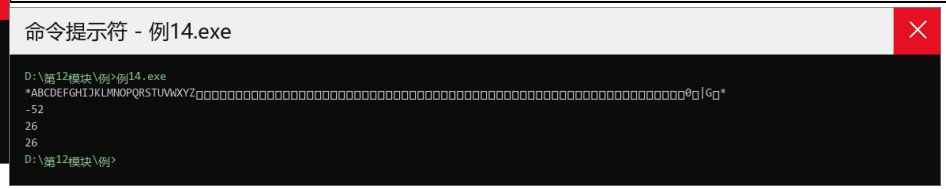
输出的name是：ABCDEFGHIJKLMNOPQRSTUVWXYZ＋乱码

name[26]的值是：-52

gcount()的值是：26

tellg()的值是：26

说明：in.read()读入时，是读到指定字节数（或文件尾）停止，不在数组最后加入一个'\0'。



综合左右：gcount()仅对read()方式读时有效，可返回最后读取的字节数；tellg()则对两种读入方式均有效。

§12. 输入输出流 - C++方式的文

已作答

例15：比较read()读超/不超过文件长度时的区别，并观察gcount()/tellg()/good()的返回值

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[30] = "000000000000000000000000000000";
    in.read(name, 20);
    cout << '*' << name << '*' << endl;
    cout << int(name[20]) << endl;
    cout << in.gcount() << endl;
    cout << in.tellg() << endl;
    cout << in.good() << endl;
    in.close();

    return 0;
}
```

Windows下运行，文件大小：26

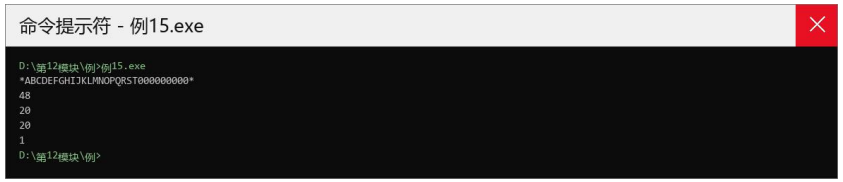
输出的name是：ABCDEFGHIJKLMNOPQRST00000000

name[20]的值是：48

gcount()的值是：20

tellg()的值是：20

good()的值是：1




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[30] = "000000000000000000000000000000";
    in.read(name, 200);
    cout << '*' << name << '*' << endl;

    cout << in.gcount() << endl;
    cout << in.tellg() <<endl;
    cout << in.good() << endl;
    in.close();

    return 0;
}
```

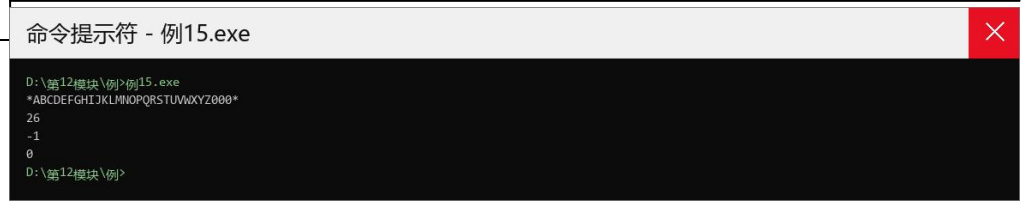
Windows下运行，文件大小：26

输出的name是：ABCDEFGHIJKLMNOPQRSTUVWXYZ000

gcount()的值是：26

tellg()的值是：-1

good()的值是：0



§12. 输入输出流 - C++方式的文

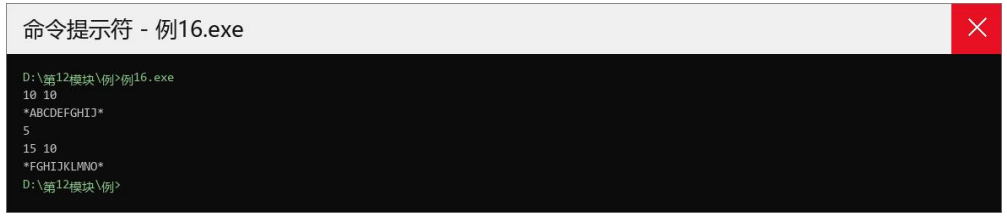
已作答

例16：使用seekg()移动文件指针，观察gcount()/tellg()/seekg()在不同情况下的返回值

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[80];
    in.read(name, 10);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[10] = '\0';
    cout << '*' << name << '*' << endl;
    in.seekg(-5, ios::cur);
    cout << in.tellg() << endl;
    in.read(name, 10);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[10] = '\0';
    cout << '*' << name << '*' << endl;
    in.close();
    return 0;
}
```

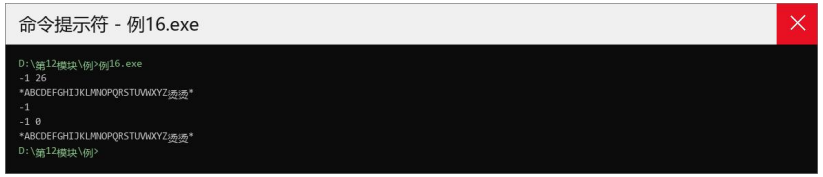
Windows下运行，输出依次是：10 10
ABCDEFGHIJ
5
15 10 / *FGHIJKLMNO*




```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[80];
    in.read(name, 30);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    in.seekg(5, ios::beg);
    cout << in.tellg() << endl;
    in.read(name, 30);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    in.close();
    return 0;
}
```

Windows下运行，输出依次是：-1 26
ABCDEFGHIJKLMNOPQRSTUVWXYZ烫烫
-1
-1 0 / *···*



综合左右：tellg()/gcount()/seekg()仅在流状态正常（good() 为真）情况下返回正确值，因此，每次操作完成后，最好判断流对象自身状态，正确才可继续下一步。

§12. 输入输出流 - C++方式的文

已作答

例17：使用seekg()/gcount()/tellg()/good()后判断流对象状态是否正确，若不正确则恢复正确状态后再继续使用

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    ifstream in("out.txt", ios::in | ios::binary);
    char name[80];
    in.read(name, 30);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    if (!in.good())
        in.clear();

    in.seekg(5, ios::beg);
    cout << in.tellg() << endl;
    in.read(name, 30);
    cout << in.tellg() << " " << in.gcount() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    if (!in.good())
        in.clear();
    in.close();
    return 0;
}
```

Windows下运行，输出依次是：-1 26
ABCDEFGHIJKLMNOPQRSTUVWXYZ烫烫
5
-1 21 /
FGHIJKLMNOPQRSTUVWXYZVWXYZ烫烫

说明：clear() 之后 seekg(5) 生效；随后 read 又读到文件尾，tellg() 变回 -1、gcount() = 21。



§12. 输入输出流 - C++方式的文

已作答

例18：读写方式打开时的seekg()/seekg()同步移动问题

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    fstream file("out.txt", ios::in|ios::out|ios::binary);
    char name[80];
    file.read(name, 30);
    cout << file.tellg() << " " << file.gcount()
                         << " " << file.tellp() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    if (!file.good())
        file.clear();

    file.seekg(5, ios::beg);
    cout << file.tellg() << " " << file.tellp() << endl;

    file.seekp(12, ios::beg);
    cout << file.tellg() << " " << file.tellp() << endl;

    strcpy(name, "abcdefghijklmnopqrstuvwxyz0123");
    file.write(name, 30);
    cout << file.tellg() << " " << file.tellp() << endl;
    file.close();

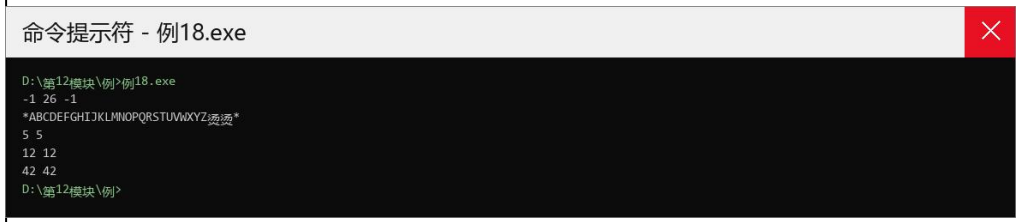
    return 0;
}
```

Windows下运行，输出依次是：-1 26 -1
ABCDEFGHIJKLMNOPQRSTUVWXYZ烫烫
5 5
12 12
42 42

结论：

1、读写方式打开时，tellg()/tellp()均可以使用，且读写后两个函数的返回值均相同

2、文件指针的移动，seekg()/seekp()均可



§12. 输入输出流 - C++方式的文

已作答

例19：读写方式打开时加ios::app方式后，读写指针移动及写入问题

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    fstream file("out.txt", ios::in|ios::out|ios::binary|ios::app);
    char name[80];
    file.read(name, 30);
    cout << file.tellg() << " " << file.gcount()
                         << " " << file.tellp() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    if (!file.good())
        file.clear();

    file.seekg(5, ios::beg);
    cout << file.tellg() << " " << file.tellp() << endl;

    file.seekp(12, ios::beg);
    cout << file.tellg() << " " << file.tellp() << endl;

    strcpy(name, "abcdefghijklmnopqrstuvwxyz0123");
    file.write(name, 30);
    cout << file.tellg() << " " << file.tellp() << endl;
    file.close();
    return 0;
}
```

Windows下运行，输出依次是：-1 26 -1
ABCDEFGHIJKLMNOPQRSTUVWXYZ烫烫
5 5
12 12
56 56

结论：

1、加ios::app后，虽然seekg()/seekp()可以移动文件指针，
但是写入的位置始终在文件末尾（追加）

2、自行测试ofstream方式打开加ios::app的情况，
与本例的结论一致(一致/不一致)



§12. 输入输出流 - C++方式的文

已作答

例20：读写方式打开时加ios::app方式后，读写指针移动及写入问题

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    fstream file("out.txt", ios::in|ios::out|ios::binary|ios::app);
    char name[80];
    file.read(name, 30);
    cout << file.tellg() << " " << file.gcount()
                         << " " << file.tellp() << endl;
    name[30] = '\0';
    cout << '*' << name << '*' << endl;
    if (!file.good())
        file.clear();

    file.seekg(5, ios::beg);
    cout << file.tellg() << " " << file.tellp() << endl;

    strcpy(name, "abcdefghijklmnopqrstuvwxyz0123");
    file.write(name, 30);
    cout << file.tellg() << " " << file.tellp() << endl;
    file.close();

    return 0;
}
```

Windows下运行，输出依次是：-1 26 -1
ABCDEFGHIJKLMNOPQRSTUVWXYZ烫烫
5 5
56 56

结论：加ios::app后，读写方式打开时，tellg()/tellp()均可以使用，且无论读写，两个函数的返回值均相同，表示两个文件指针是同步移动的



§12. 输入输出流 - C++方式的文

已作答

例21：不同打开方式下文件指针的初始值问题

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    cout << "请查看当前out.txt文件的大小" << endl;
    system("pause");

    out.open("out.txt", ios::out | ios::app);
    cout << out.tellp() << endl;
    out << "0123456789";
    cout << out.tellp() << endl;
    out.close();

    return 0;
}
```

Windows下运行，
1、执行到system("pause")的时候，out.txt的大小是：26
2、加ios::app后，写方式打开，tellp()为0，
写入是在文件结束(开始/结束)位置，
完成后tellp()是36



§12. 输入输出流 - C++方式的文

已作答

例22：不同打开方式下文件指针的初始值问题

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    cout << "请查看当前out.txt文件的大小" << endl;
    system("pause");

    out.open("out.txt", ios::out | ios::ate);
    cout << out.tellp() << endl;
    out << "0123456789";
    cout << out.tellp() << endl;
    out.close();

    return 0;
}
```

Windows下运行，
1、执行到system("pause")的时候，out.txt的大小是：26
2、加ios::ate后，写方式打开，tellp()为0，
　　写入是在文件开始(开始/结束)位置，
　　完成后tellp()是10
注：ate = at end



§12. 输入输出流 - C++方式的文

已作答

例23：不同打开方式下文件指针的初始值问题

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    cout << "请查看当前out.txt文件的大小" << endl;
    system("pause");

    out.open("out.txt", ios::out | ios::ate | ios::app);
    cout << out.tellp() << endl;
    out << "0123456789";
    cout << out.tellp() << endl;
    out.close();

    return 0;
}
```

Windows下运行，
1、执行到system("pause")的时候，out.txt的大小是：26
2、同时加ios::ate|ios::app后，写方式打开，tellp()为26，
写入是在文件结束(开始/结束)位置，
完成后tellp()是36
结论：结合本例及前两例，ios::ate加在ofstream方式的输出文件上无(有/无)实用价值



§12. 输入输出流 - C++方式的文

已作答

例24：不同打开方式下文件指针的初始值问题

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    cout << "请查看当前out.txt文件的大小" << endl;
    system("pause");

    ifstream in("out.txt", ios::in);
    cout << in.tellg() << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，
1、执行到system("pause")的时候，out.txt的大小是：26
2、正常读方式打开，tellg()和peek()为0和65，
表示从文件的开始(开始/结束)位置读



§12. 输入输出流 - C++方式的文

已作答

例25：不同打开方式下文件指针的初始值问题

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char *argv[])
{
    ofstream out("out.txt", ios::out);
    out << "ABCDEFGHIJKLMNOPQRSTUVWXYZ"; //无换行符
    out.close();

    cout << "请查看当前out.txt文件的大小" << endl;
    system("pause");

    ifstream in("out.txt", ios::in | ios::ate);
    cout << in.tellg() << endl;
    cout << in.peek() << endl;
    in.close();

    return 0;
}
```

Windows下运行，
1、执行到system("pause")的时候，out.txt的大小是：26
2、加ios::ate后，读方式打开，tellg()和peek()为26和-1，
表示从文件的结束(开始/结束)位置读

结论：
1、结合本例及上例，ios::ate加在ifstream方式的输出文件上
有(有/无)实用价值
2、为了避免细节记忆错误，另一种做法是，舍弃ios::ate特性不同，
在需要读写时直接用seekg()/seekp()自行移动文件开头/结尾，
你是否赞成(赞成/反对)这种做法

